

Documentación Técnica y Arquitectura: KeySearch V 4.0

1. Visión General del Proyecto

KeySearch V 4.0 es una herramienta automatizada de inteligencia SEO diseñada para extraer, filtrar, categorizar y priorizar intenciones de búsqueda reales de los usuarios en Google. Su objetivo principal es descubrir qué preguntan las personas sobre un tema específico, analizar su volumen e interés histórico, y entregar un reporte estructurado y priorizado en formato Excel.

A diferencia de los scrapers convencionales que dependen de navegadores pesados (como Selenium), KeySearch V 4.0 utiliza una arquitectura "Browserless" (HTTP puro), lo que lo hace exponencialmente más rápido, estable y resistente a los bloqueos (CAPTCHAs) de Google.

2. Arquitectura y Flujo de Trabajo (Pipeline)

El sistema procesa la información en las siguientes fases consecutivas:

1. Ingesta y Contexto (`main.py` y `config.py`)
2. Extracción Browserless (`scraper/autocomplete.py` y `scraper/google_serp.py`)
3. Filtro de Inteligencia Artificial (`scraper/ai_filter.py`)
4. Categorización Automática (`scraper/categorizer.py`)
5. Estimación y Priorización (`scraper/volume_estimator.py` y `scraper/google_ads_metrics.py`)
6. Exportación Estilizada (`exporters/excel_export.py`)

Diagrama Lógico Simplificado

Input de Usuario → Consultas HTTP a Google → Extracción (Sugerencias, PAA, etc.) → Filtro IA (Groq) → Categorización Local → Cruce de Datos (Trends & Ads) → Reporte Excel V4

3. Explicación Detallada de Módulos y Funciones

3.1. Ingesta y Contexto (`main.py` & `config.py`)

Es el punto de entrada de la aplicación.

- `mostrar_banner()`: Renderiza el título "KeySearch V 4.0" en la consola usando la librería `rich`.
- `_solicitar_contexto_busqueda()`: Recibe el país objetivo (ej. Colombia, España, El Salvador) y lo normaliza usando un diccionario de alias (`COUNTRY_ALIASES` en `config.py`) para evitar errores tipográficos y asociar el código ISO correcto (ej. "sv" o

"El Salvador" se transforman en las directivas correctas para Google Ads y Trends).

3.2. Motor de Extracción HTTP (**scraper/**)

Este es el corazón de la recolección de datos, operando sin abrir ningún navegador.

- **get_autocomplete_suggestions(keyword):**
 - Archivo: `autocomplete.py`
 - Función: Ataca directamente la API oculta de autocompletado de Firefox/Google (`suggestqueries.google.com`). Extrae las sugerencias que Google le da a los usuarios a medida que escriben. Al usar iteraciones con el alfabeto (ej. "keyword a", "keyword b"), extrae cientos de sugerencias en segundos.
- **get_question_suggestions(keyword):**
 - Archivo: `autocomplete.py`
 - Función: Combina la palabra clave con modificadores interrogativos ("qué", "cómo", "cuándo") y las envía a la API de autocompletado para descubrir las dudas exactas que tiene la gente.
- **scrape_google(keyword):**
 - Archivo: `google_serp.py`
 - Función: Hace una petición HTTP directa (simulando ser un usuario real mediante rotación de *User-Agents*) a la página de resultados de Google (SERP). Luego, usa `BeautifulSoup` para parsear el código HTML y extraer dos elementos clave:
 1. PAA (People Also Ask / Otras preguntas de los usuarios).
 2. Búsquedas Relacionadas que aparecen al final de la página.

3.3. Filtrado por Inteligencia Artificial (**scraper/ai_filter.py**)

- **filtrar_con_ia(keywords, keyword_base, pais):**
 - Función: Recibe las listas "crudas" de palabras extraídas y se conecta a la **API de Groq** usando el modelo ultra-rápido `llama-3.3-70b-versatile`.
 - Mecanismo: Se le inyecta un *Prompt* estricto ordenándole que actúe como analista SEO. La IA analiza semánticamente cada palabra clave y elimina las que sean basura o que correspondan a ubicaciones geográficas incorrectas (ej. si el país objetivo es Colombia, elimina automáticamente sugerencias que contengan "El Salvador" o "México").
 - Retorna un Array JSON limpio, protegiendo al sistema de "alucinaciones" al verificar que las palabras devueltas existían en la lista original.

3.4. Categorización Automática (**scraper/categorizer.py**)

- **auto_categorizar(keyword):**
 - Función: Reemplaza el antiguo sistema manual donde el usuario debía teclear la categoría.
 - Mecanismo: Posee una taxonomía (diccionario) embebida con cientos de palabras clave predefinidas por sectores (Salud, Legal, Mascotas, etc.). Utiliza Expresiones Regulares (`regex` con "Word Boundaries" `\b`) para buscar coincidencias exactas y

de plurales. Si la keyword es "perros", la asigna a "Mascotas". Si es "villa de leyva", reconoce que "leyva" no es la palabra legal "ley" y la clasifica apropiadamente, evitando falsos positivos.

3.5. Cruce de Datos y Priorización (`scraper/volume_estimator.py` & `google_ads_metrics.py`)

- `estimar_volumenes(...)`:

- Función: Recibe todas las listas limpias. En lugar de inventar números aleatorios, asigna un "Score de Prioridad" basado en la posición en la que Google arrojó el resultado (una palabra que aparece de primera en el autocompletado tiene un score más alto que la décima).
- **Google Trends**: Usa la librería `pytrends` para enviar las palabras prioritarias a Google Trends y obtener el interés histórico (0-100) en los últimos 12 meses.

- `enrich_with_google_ads_metrics(...)`:

- Función: Si el usuario tiene configurada su cuenta y credenciales en `google-ads.yaml`, esta función cruza la base de datos limpia con la API oficial de Google Ads, descargando el volumen de búsqueda mensual promedio y el costo por clic (Bid alto/bajo).

3.6. Exportación y Diseño (`exporters/excel_export.py`)

- `exportar_excel(keyword, datos)`:

- Función: Genera el documento final interactivo usando la librería `openpyxl`.
- Mecanismo de Diseño: Este módulo fue programado para **clonar a la perfección la "PLANTILLA" corporativa**:
 - **Tipografía**: Todo el documento usa `Century Gothic`.
 - **Colores**: Se aplica el tono azul naval exacto (002060) para los encabezados.
 - **Estructura "Resumen"**: Genera una hoja principal con la distribución de prioridad ("Muy alta", "Alta", etc.) consolidada en dos columnas para una lectura ejecutiva perfecta.
 - **Datos trazables**: Separa en pestañas las Sugerencias, PAA, y Relacionadas, con filas de colores intercalados para lectura limpia y sin resaltar columnas geográficas que rompían el formato original.

4. Empaquetado (`BuscadorTendenciasGoogle.spec`)

Para asegurar que cualquier persona pueda correr la herramienta sin instalar Python ni dependencias complejas, el proyecto completo (incluyendo dependencias como `BeautifulSoup`, `openpyxl`, `pytrends`, y `requests`) se compila en un archivo `.exe` independiente a través de **PyInstaller**. Todo el pipeline y la estructura de archivos quedan encapsulados en un solo ejecutable ligero.